HIGHER EDUCATION PROGRAMMES

Name:	Tallulah Schreiber	**Student Number:**	27HA2603309
Support Centre:	Gqeberha	**Module Name:**	Creative Project: Game Production
Due Date:	11 September 2026	**Module Code:**	HCGDP1181
Submission Date:	08 September 2026	**Assessment:**	*FA1*

Submission Checklist: Mark the following with an **x** once you have confirmed you have done it.

• Check you are using Unity Version 6000.0.63f1.	☒
• Upload your project to GitHub using GitHub Desktop. (HFGE1181 Unit 2.6)	☒
• I have committed all changes by the due date.	☒
• I have pushed my changes to my repository.	☒
• I have set my GitHub Repository to public.	☒
• I have ensured all adjoining documents and files required for this assessment have been uploaded to my GitHub repository. *(e.g. Builds, GDD's, Daily Reports)*	☒
• I have confirmed myself that all files are present and are my most up to date version.	☒
• I have confirmed that the link I am supplying works and leads to the project for this submission.	☒

Repository Link:	https://github.com/tallulahschreiber27/Tallulah_Schreiber_27HA2603309_HCGDP1181_POE

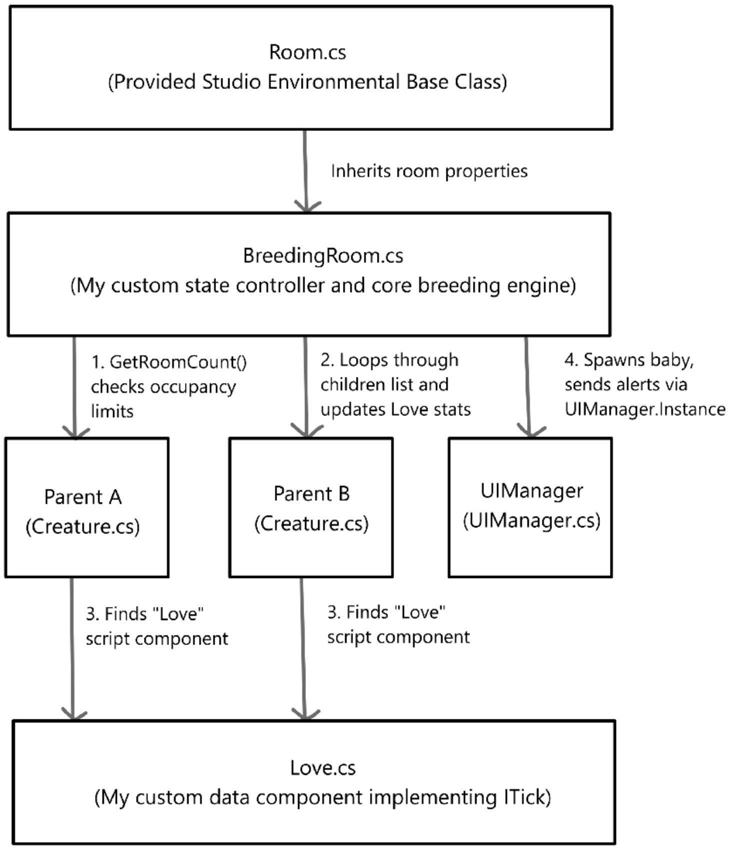
SECTION A: Game Proposal

1. Design Approach & System Architecture

The goal of this system is to make breeding feel natural and fun for the player, just like in cozy games like *Slime Rancher*. Instead of making players click boring menus or buttons, the system works automatically in the background using the creature's own stats.

The system relies on a single main script called BreedingRoom.cs that attaches directly to the room in Unity. This script keeps the game running smoothly by constantly checking how many creatures are inside. If the room gets overcrowded, the script automatically steps in to freeze the breeding process until it is safe again.

Diagram:


Room.cs
(Provided Studio Environmental Base Class)
Inherits room properties
BreedingRoom.cs
(My custom state controller and core breeding engine)
1. GetRoomCount()
checks occupancy
limits
2. Loops through
children list and
updates Love stats
4. Spawns baby,
sends alerts via
UIManager.Instance
Parent A
(Creature.cs)
Parent B
(Creature.cs)
UIManager
(UIManager.cs)
3. Finds "Love"
script component
3. Finds "Love"
script component
Love.cs
(My custom data component implementing ITick)

2. Layman's Explanation (How the Code Works)

The breeding room operates through a simple four-step process:

1. **The Sensor:** The room watches its doors. The exact moment **two creatures** walk inside together, it connects to their stats and starts the romance cycle.
2. **The Love Tracker:** While the two creatures are alone in the room, their custom **Love** stat begins to climb automatically from 0 to 100. The player can watch this number grow in real-time on their screen panel.
3. **The Crowding Guard:** If a third creature wanders into the room before the bar hits 100, the room instantly stops generating love points. It also pops up a warning message on the player's screen saying *"Breeding Room Blocked!"* so the player knows to remove the extra creature.
4. **The Birth & Cooldown:** As soon as the love bar hits 100, the parents' love stats reset back to 0. A 10-second rest cooldown triggers so they don't breed instantly again, and a brand-new baby hatches! The baby inherits stats from both parents, gets a unique name tag, and has a 10% chance to mutate into a unique color or get rare body parts (like horns).

3. Research Justification & Industry References

This design takes inspiration from two legendary virtual pet and simulation games:

- **Natural Life Cycles (*Tamagotchi*):** In classic *Tamagotchi* games, pets breed naturally when they spend time together. Our automated room uses this approach, letting the game progress smoothly based on where the creatures are rather than forcing the player to click through menus.
- **Mixing Stats & Mutations (*Monster Rancher*):** Our inheritance system is inspired by *Monster Rancher*. Instead of just making an exact copy of a parent, the code mixes the stats of both parents to create a unique baby. We also added a small chance for random visual mutations (like rogue color shifts or mesh swaps) to keep the gameplay surprising and rewarding.
- **References:**
  - *Bandai Namco Virtual Pet Design Archives:* Tamagotchi Patterns
  - *Tecmo Koei Simulation Game Mechanics:* Monster Rancher Combining Archives
  - *Unity Technologies C# Scripting Reference:* Unity Component Documentation

4. Production Timeline & Milestone Grid

Milestone ID	Task focus	Dates	Hours	Outcome
Milestone 1	Project planning	August 29 – August 31	4 hours	Explore provided unity scene environment
Milestone 2	Custom breeding system	September 1 – September 3	2 hours	Implement breeding system
Milestone 3	Refining breeding system (add breeding room + love trait)	September 4 – September 6	3 hours	Polish the breeding system into what I truly want.
Milestone 4	Change method of breeding	September 7 – September 8	4 hours	The creatures wait for the love meter to complete before they can breed.
Milestone 5	Mutation polish	September 8 – September 9	2 hours	Implement mutation system.
Milestone 6	Finishing touches	September 10	2 hours	Prepare for submission.